Programming Fundamentals

Week 7 Revision — Files and JSON

Programming Fundamentals Team

SETU

2026-11-06

Today — A Quick Week 7 Recap

- What is a file?
- Why do we save data?
- How do we open files in Python?

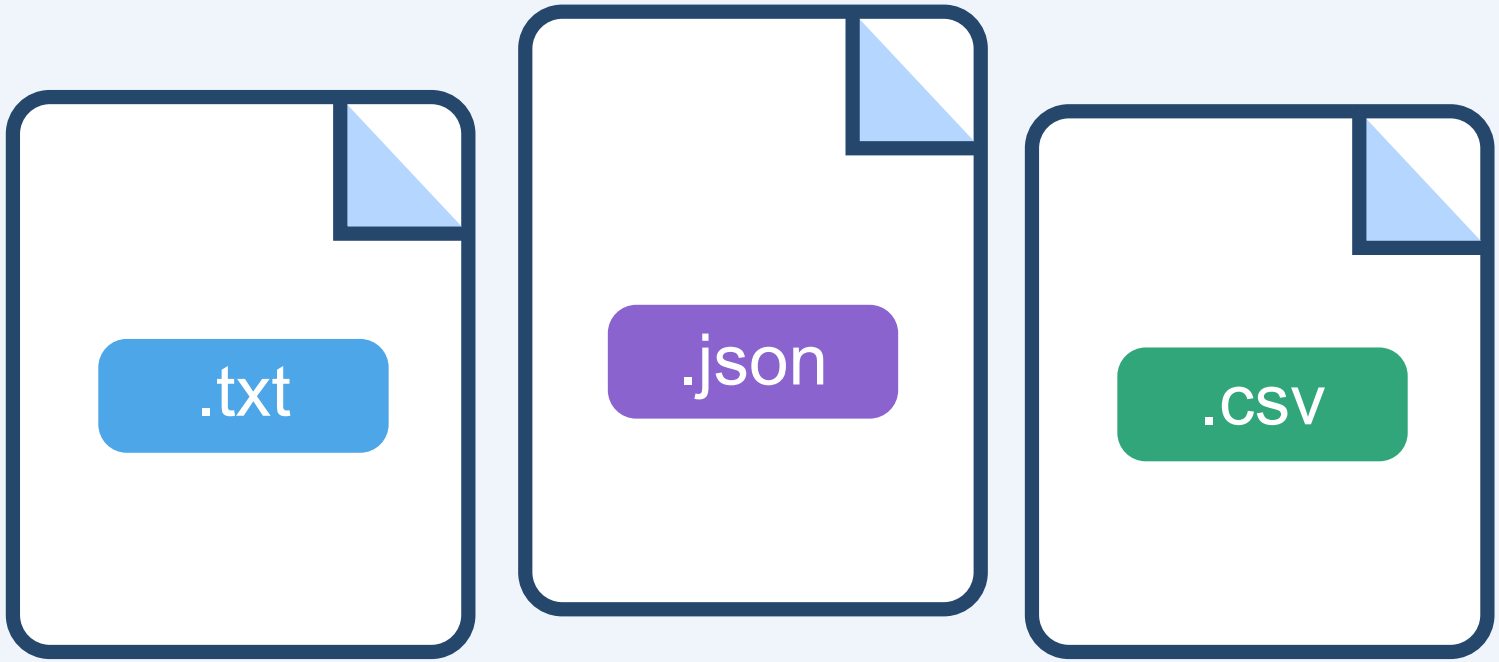
What Is a File?

A **file** is data stored on a computer so we can use it again later.

Examples:

- `notes.txt` — plain text
- `scores.json` — structured data
- `results.csv` — rows and columns

What Is a File?



Why Save Data to a File?

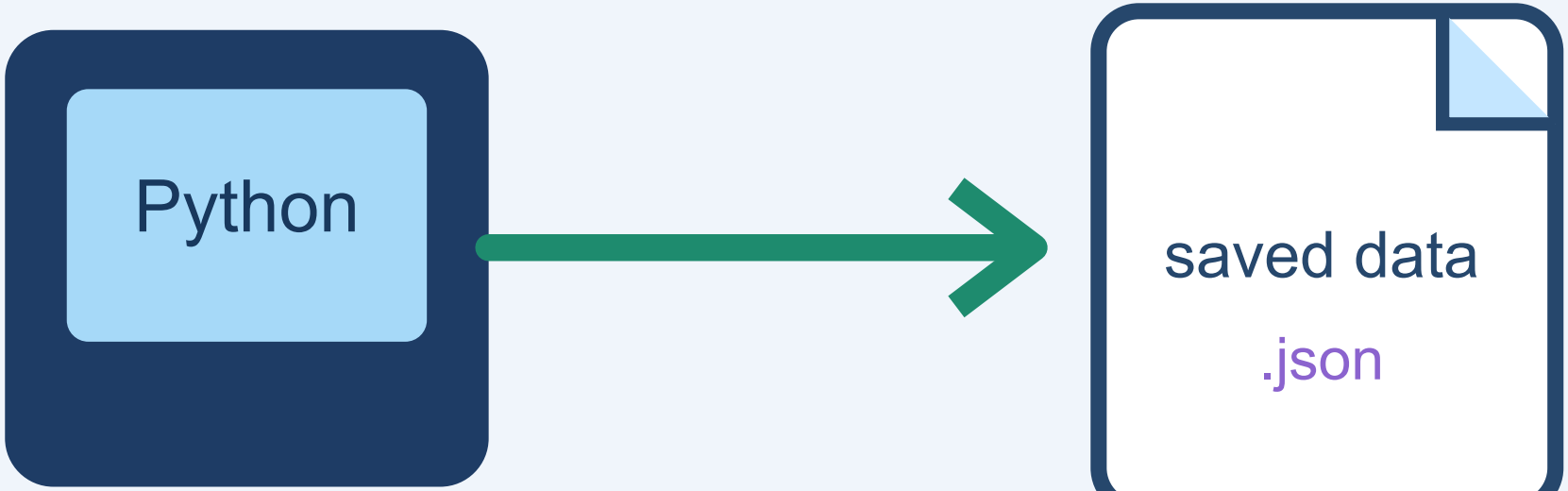
Without a file, data stored in variables is normally lost when the program ends.

With a file, we can:

- save information
- close the program
- load it again next time

This is called persistence.

Why Save Data to a File?

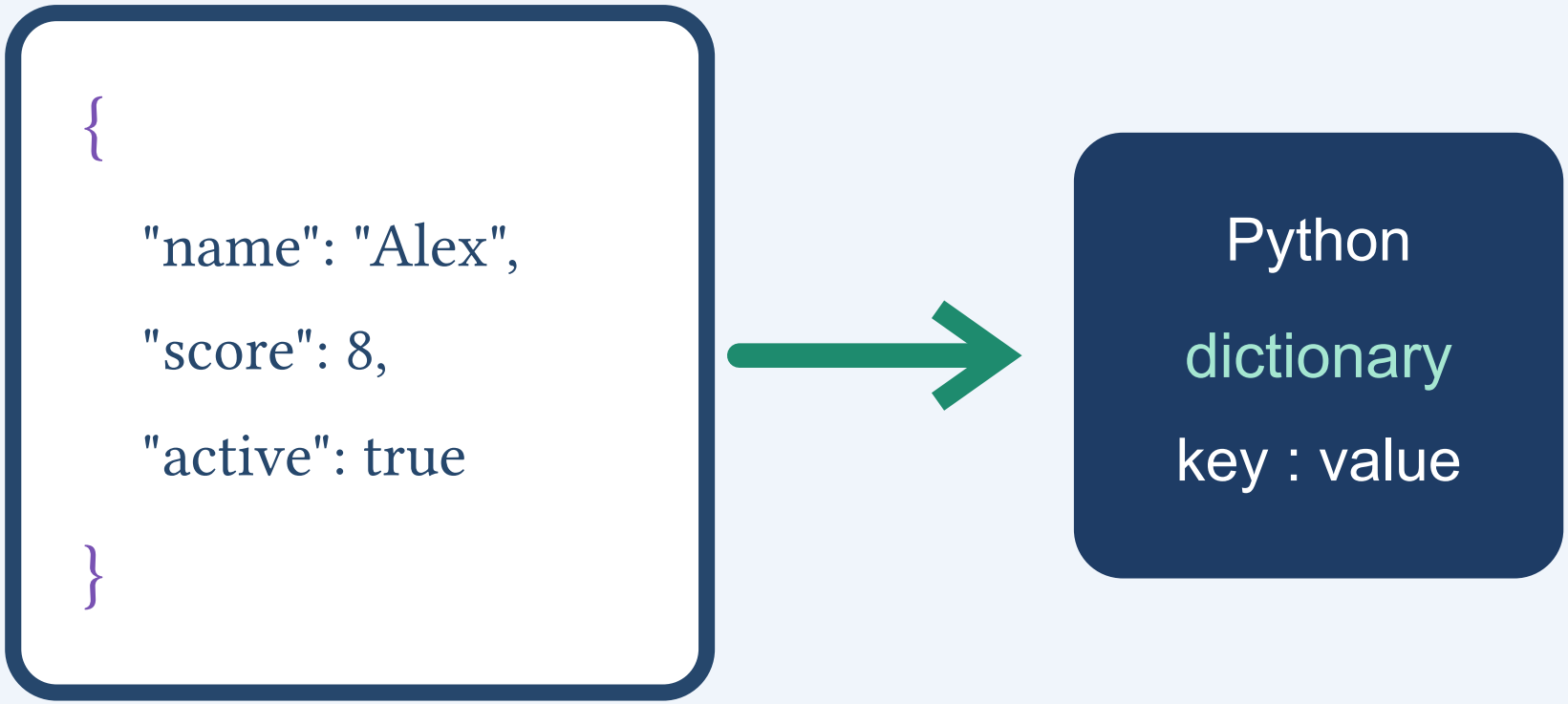


Why Use JSON?

JSON stores **structured data** using keys, values, lists and objects.

- easy for people to read
- works well with Python dictionaries and lists
- separates stored data from program logic
- can be updated without editing Python code

Why Use JSON?



A Simple JSON File

`student.json` contains:

```
{
  "name": "Alex",
  "score": 8,
  "subjects": ["Python", "Maths"]
}
```

Question: Which values are a string, a number and a list?

Ways to Open a File

Mode	What it does	Example
`"r"`	Read an existing file. Fails if missing.	`open("notes.txt", "r")`
`"w"`	Write. Creates or **overwrites** a file.	`open("notes.txt", "w")`
`"a"`	Append to the end. Creates if missing.	`open("notes.txt", "a")`
`"x"`	Create a new file. Fails if it exists.	`open("notes.txt", "x")`



Reading a Text File — "r"

Imagine `notes.txt` already contains `Remember your lab!`.

```
with open("notes.txt", "r", encoding="utf-8") as file:
    content = file.read()

print(content)
```

Output: `Remember your lab!`

`with` closes the file automatically.

Writing a Text File — "w"

```
with open("notes.txt", "w", encoding="utf-8") as file:
    file.write("Hello from Python!\n")
```

- Creates `notes.txt` if it does not exist.
- **Warning:** replaces its existing contents if it does exist.
- `\n` starts a new line.

Appending to a Text File — "a"

```
with open("notes.txt", "a", encoding="utf-8") as file:
    file.write("One more note.\n")
```

- Adds text to the **end** of the file.
- Keeps the text already there.
- Creates the file if needed.

Saving a Dictionary as JSON

```
import json

student = {"name": "Alex", "score": 8}

with open("student.json", "w", encoding="utf-8") as file:
    json.dump(student, file, indent=4)
```

`json.dump()` writes Python data to a JSON file.

Loading Data from JSON

```
import json

with open("student.json", "r", encoding="utf-8") as file:
    student = json.load(file)

print(student["name"])
```

Output: `Alex`

`json.load()` reads JSON into Python data (here, a dictionary).

Remember the Working Directory

`open("notes.txt")` looks relative to the **current working directory**.

```
import os
print(os.getcwd())
```

For project files, we can build a path relative to our script:

```
from pathlib import Path
path = Path(__file__).resolve().parent / "notes.txt"
```

Quick Check — Can You Explain?

1. Why save data in a file?
2. What is JSON useful for?
3. Which mode adds text without deleting existing text?
4. What is the danger of `"w"`?
5. What does `with` do for us?